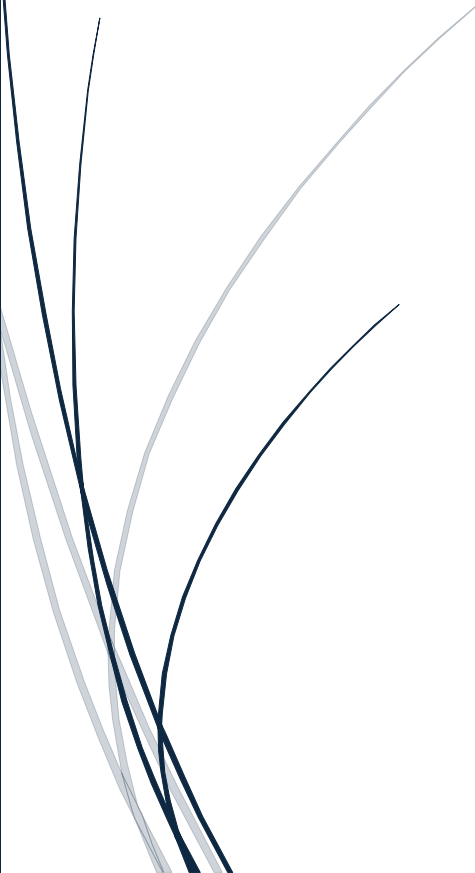


Proyecto Final

CROOSFITBOX

Curso SQL - Coderhouse



Emiliana María Torres
CODERHOUSE

INFORME

Introducción

En este documento se presenta la información correspondiente a la base de datos diseñada para su implementación en un gimnasio o box de CrossFit. En ella se gestionan clases de Crossfit, entrenamiento Funcional y Weightlifting. Además, se incluye un análisis descriptivo con una tabla que muestra el contenido detallado de la base de datos, junto con un diagrama esquemático que representa las relaciones entre sus entidades.

Situación problemática

En un gimnasio o box especializado en CrossFit, Funcional y Weightlifting, la gestión de forma manual de las inscripciones, reservas de clases, asignación de entrenadores y el control de pagos genera demoras, errores y falta de seguimiento. Por lo tanto, esta situación dificulta tanto el trabajo del personal como la experiencia del cliente.

Por ello, surge la necesidad de diseñar una base de datos que permita centralizar la información, mantener un registro confiable de las operaciones diarias y optimizar la administración del servicio, evitando inconvenientes que puedan surgir por la falta de control o la duplicación de información.

Objetivos

El objetivo principal es mejorar la performance del manejo del box, con el fin de tener todo organizado y plasmado en una base de datos.

Implementar una base de datos que ordene y automatice los procesos operativos del box, facilitando la gestión de clases, entrenadores y pagos.

Tener un sistema estructurado y eficiente para mejorar el rendimiento administrativo, asegurar un seguimiento detallado de las actividades y garantizar una experiencia satisfactoria tanto para los clientes como para el personal del gimnasio.

Diagrama Entidad Relación (DER)

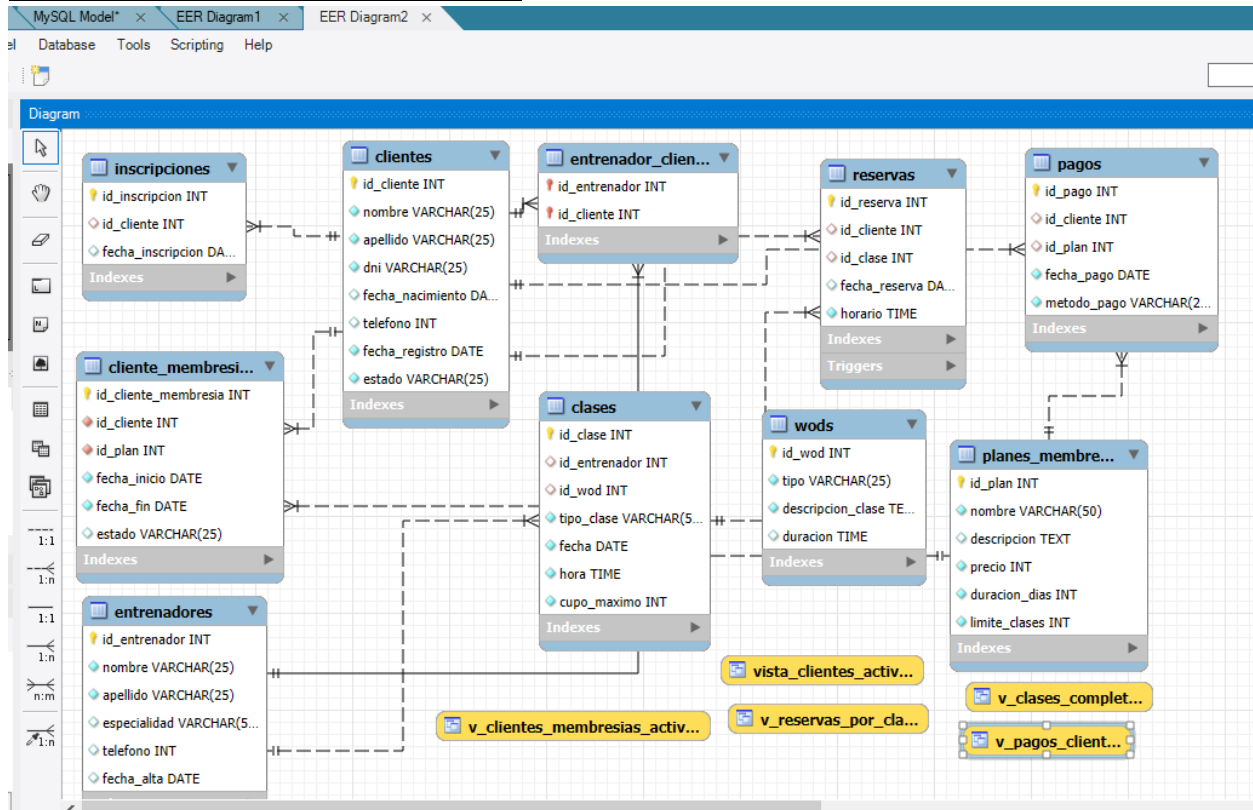


Diagrama de Tablas

Tabla	Campos	Tipo de Dato	Detalle Campo	PK	FK
clientes	Id_cliente	INT	Identificador único	PK	
	Nombre	VARCHAR	Nombre del socio		
	Apellido	VARCHAR	Apellido del socio		
	Dni	VARCHAR	Documento de identidad		
	Fecha_nacimiento	DATE	Fecha de Nacimiento		
	Telefono	INT	Teléfono de contacto		
	Fecha_registro	DATE	Fecha de registro del cliente		

	estado	VARCHAR	Estado del socio (Activo/Inactivo)		
Entrenadores	Id_entrenador	INT	Identificador único	PK	
	Nombre	VARCHAR	Nombre del entrenador		
	Apellido	VARCHAR	Apellido del entrenador		
	Especialidad	VARCHAR	Especialidad del entrenador		
	Telefono	INT	Teléfono del entrenador		
	Fecha_alta	DATE	momento en que comenzó a cotizar		
wods	Id_wod	INT	Identificador único	PK	
	Tipo	VARCHAR	Tipo de clase, ej: AMRAP, EMOM, FOR TIME,etc		
	Descripcion_clase	TEXT	Descripción detallada de ejercicios a realizar en la clase en el warmup, weightlifting y wod.		
	Duracion	TIME	Tiempo que lleva el wod		
Clases	Id_clase	INT	Identificador único	PK	
	Id_entrenador	INT	Identificador del entrenador		FK
	Id_wod	INT	Identificadordel wod		FK
	Tipo_clase	VARCHAR	Ej: crossfit, funcional, weightlifting		
	Fecha	DATE	Días que muestran las clases disponibles (ej:crossfit, funcional o weightlifting) de lunes a sábados.		
	Hora	TIME	Horarios de las clases		

	Cupo_maximo	INT	Límite de personas por clase		
Inscripciones	Id_inscripcion	INT	Identificador único	PK	
	Id_cliente	INT	Identificador del cliente		FK
	Fecha_incripcion	DATE	Fecha que se inscribió el cliente al box por primera vez		
reservas	Id_reserva	INT	Identificador único	PK	
	Id_cliente	INT	Identificador del cliente		FK
	Id_clase	INT	Identificador de la clase		FK
	Fecha_reserva	DATE	Día de la semana para anotarse de Lunes a Sábados		
	Horario	TIME	Horarios de las clases para anotarse		
Planes_membresias	Id_plan	INT	Identificador único	PK	
	Nombre	VARCHAR	Nombre de la membresía, ej: básico, premium, ilimitado		
	Descripcion	TEXT	Descirpción de lo que implica un pack de básico, premium e ilimitado, corerspondiendo a 8 o 12 clases, o ilimitado.		
	Precio	INT	Precio de cada pack		
	Duracion_dias	INT	Tiempo que dura el pack (1 mes)		
	Limite_clases	INT	8, 12, 100 en caso de que sea ilimitado		

pagos	Id_pago	INT	Identificador único	PK	
	Id_cliente	INT	Identificador del cliente		FK
	Id_membresia	INT	Identificador membresía		FK
	Id_cliente_membresia	INT	Identificador cliente membresía		FK
	Fecha_pago	DATE	Fecha en que realizó el pago		
	Metodo_pago	VARCHAR	Forma en la que realizó el pago (efectivo, MP, débito)		
entrenador_cliente	id_entrenador	INT	Identificador del entrenador	PK	FK
	id_cliente	INT	Identificador del cliente	PK	FK
Cliente_membresias	Id_cliente_membresia	INT	Identificador cliente membresia	PK	
	Id_cliente	INT	Identificador cliente		FK
	Id_plan	INT	Identificador del plan		FK
	Fecha_inicio	DATE	Fecha inicio membresía		
	Fecha_fin	DATE	Fecha fin membresía		
	Estado	VARCHAR	Activo o Inactivo		

VISTAS

1. Vista_clientes_activos

El objetivo de esta vista es ver rápidamente los clientes que están habilitados. Nos muestra: Id_cliente, nombre, apellido, dni, fecha_registro.

2. Vista_clases_completas

El objetivo es mostrar toda la info relevante de una clase, nos puede servir como una agenda diaria. Unimos las tablas clases, entrenadores y wods con un JOIN.

De la tabla clases usamos las columnas id_clase, fecha, hora, tipo_clase y cupo_maximo.

De la tabla entrenadores usamos las columnas nombre como nombre_entrenador y apellido como apellido_entrenador.

De la tabla wods usamos las columnas tipo como tipo_wod y duración.

3. Vista_reservas_por_clase

El objetivo es saber cuántos inscriptos tiene cada clase así nos sirve para controlar cupos. Necesito las tablas clases y reservas, hacer un JOIN y luego agrupar por clase con un GROUP BY; hacer un count de reservas para que me cuente el número de reservas.

Necesito de la tabla clases las columnas: id_clase, fecha, hora y tipo_clase.

De la tabla reservas uso la columna id_reserva para hacer el count.

4. Vista_clientes_membresias_activas

El objetivo es para obtener rápidamente los clientes que tienen una membresía activa, junto con los datos más importante de su plan.

Con esta vista podemos ver qué clientes pueden reservar clases, controlar su acceso al box, generar una lista de clientes activos.

Necesitamos las tablas clientes y planes_membresía. Nos sirve para hacer un seguimiento de planes activos.

De la tabla clientes usamos las columnas id_cliente, nombre, apellido, de la tabla planes_membresia usamos las columnas nombre como nombre_membresia, precio. Hacemos un JOIN de las 2 tablas usando el id_cliente, une clientes con sus membresías

5. Vista_pagos_clientes

El objetivo es ver el historial de pagos. Usamos las tablas clientes, pagos y membresías y hacemos un JOIN.

De la tabla clientes usamos las columnas nombre, apellido.

De la tabla pagos usamos las columnas id_pago, fecha_pago, método_pago.

De la tabla membresías usamos las columnas nombre como nombre_membresia, y la columna precio.

FUNCIONES

1. Calcular la edad de un cliente

Calcula la edad en años a partir de fecha_nacimiento.

2. Cantidad de clases reservadas por un cliente

Nos devuelve cuántas reservas realizó un cliente

TRIGGERS

1. Evitar sobrepasar el cupo máximo de una clase

Nos sirve porque no nos permitirá hacer reservas si la clase ya alcanzó su cupo. Usamos la tabla reservas y el tipo BEFORE INSERT ya que necesitamos que antes de que se inserte un dato en la tabla reservas, o sea antes de que un cliente pueda reservar una clase (antes que se inserte una nueva fila en la tabla reservas), no nos deje hacerlo si el cupo de esa clase está lleno. El FOR EACH ROW es que se ejecuta una vez por cada fila agregada (reserva) que se intenta insertar.

Si el trigger falla, la reserva no se guarda.

Este trigger nos garantiza la integridad del negocio porque evita que se registren más reservas que el cupo permitido así se controla que no se exceda el cupo máximo de una clase antes de registrar una reserva.

2. Activar un cliente automáticamente al registrar un pago

El objetivo de este trigger es cambiar el estado del cliente a Activo cuando realice un pago. Usamos la tabla pagos. Es de tipo AFTER INSERT porque una vez que el cliente realice el pago (se inserte un dato en la tabla pagos), recién cambia su estado de inactivo a activo. Así activa automáticamente al cliente cuando paga; se actualiza automáticamente el estado del cliente cuando se registra un pago.

STORED PROCEDURES

1. Registrar una reserva de una clase

El objetivo de este stored procedure es registrar una reserva sólo si el cliente está activo y si la clase existe.

El control de cupo ya lo hace el trigger, así que aquí no se repite. Se evita que clientes inactivos se inscriban a las clases, y asegura que la clase a la que se inscriba exista.

2. Registrar un pago de membresía

El objetivo es registrar un pago y activar automáticamente al cliente reforzando así el trigger.

El trigger AFTER INSERT ON pagos era el que se encargaba de activar al cliente. Aquí se registra el pago, inserta el pago y así dispara el trigger que activa automáticamente al cliente una vez que ya se realizó el pago.

Enlace al script SQL:

[CH_MySQL/proyecto_final at main · EmiTorres7/CH_MySQL](#)

INFORME

El informe resume indicadores clave del box mediante gráficos de torta y barras, permitiendo analizar el estado de los clientes, la popularidad de los planes y los métodos de pago más utilizados.

Resumen general

Cumple el rol de un dashboard inicial. Nos muestra:

- Clientes activos
- Clientes inactivos
- Cantidad de planes disponibles
- Pagos registrados

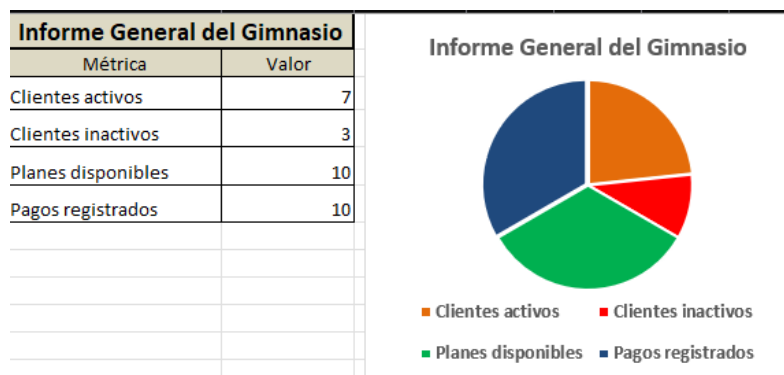


Gráfico cliente por estado

Nos permite visualizar rápidamente la proporción de clientes habilitados. Nos muestra los clientes Activos y los Inactivos

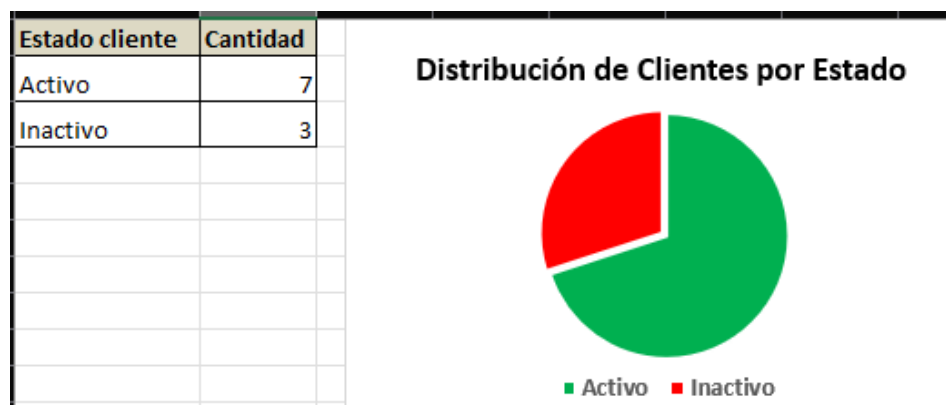


Gráfico planes de Membresías más usadas

Nos muestra cuáles son los planes más elegidos. Nos muestra los tipos de planes.

Plan	Cantidad de Clientes
Plan Básico	2
Plan Pro	3
Plan Standard	2
Otros	3

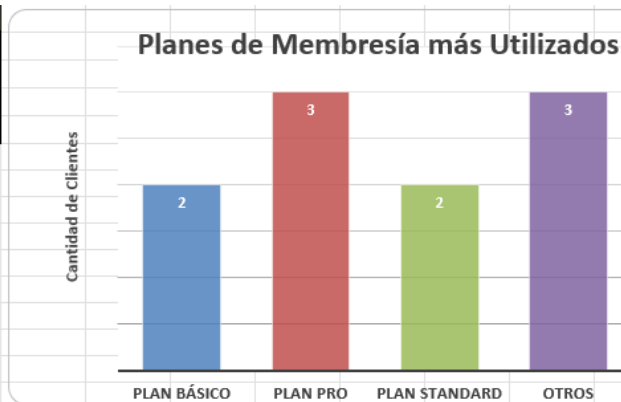
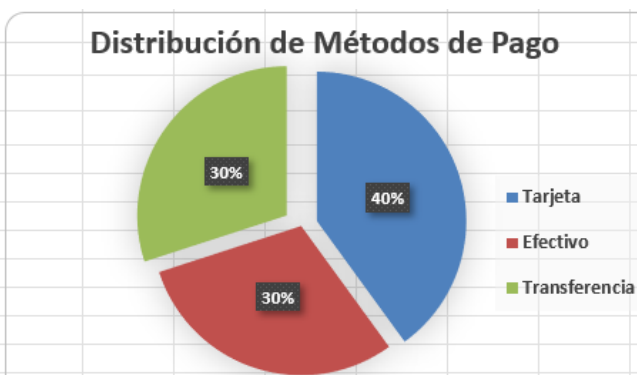


Gráfico de Métodos de pago

Esto nos informa los principales métodos de pago y las preferencias de pago del cliente.

Método	Cantidad
Tarjeta	4
Efectivo	3
Transferencia	3



Listado de Herramientas y tecnologías

- **MySQL Workbench:** utilicé esta interfaz para diseñar y crear la base de datos relacional, gestionar tablas y realizar consultas SQL.
- **Draw.io:** usé esta herramienta para diseñar el diagrama entidad-relación (DER) y visualizar la estructura y relaciones de la base de datos. Este lo comparé con el Reverse Engineer del workbench del MySQL.
- **Excel:** para cargar datos, hacer tablas y gráficos de visualización
- **GitHub:** para el control de versiones para gestionar el código, mantener un respaldo de las actualizaciones que voy realizando, y facilitar la manera de acceso de los archivos.

